

Architecture Assessment & Modernization Proposal for PWT Project

RAMESH BABU

NOVEMBER 24

2025

Architectural analysis of PWT Project for the current system, evaluation of the system for bottlenecks, pain-points and proposal of betterments.

TABLE OF CONTENTS

Contents

I.	About this Documentation	1
	Documentation Overview	1
	Scope of this Document	1
II.	Document Structure	2
1.	Overview of document structure	2
2.	Structure of Document (Sections)	2
III.	Current System: Overview	3
1.	Frontend:	3
2.	Backend:	3
3.	Databases:	3
4.	Technical Characteristics (NF):	3
IV.	Current System: Pain-points Analysis	4
A.	Technical Debts: Legacy AngularJS UI Technology	4
B.	Technical Debts: Flask as Monolithic API Layer	4
C.	Pain Point: SQL Server MAY Struggles With Scaling Read Loads	4
D.	Bottleneck: MongoDB No Clear Data Segregation	5
E.	Pain Point: Lack of Caching & No Read / Write Separation	5
V.	Technical / Architectural Solutions for each Pain-points	6
	Resolution A — Replace AngularJS with suitable UI Framework	6
	Resolution B — Break Flask Monolith to Microservices or Modular Monolith	7
	Resolution C — Read / Write Modernization	8
	Resolution D — Use MongoDB Properly for Variable Data	9
	Resolution E — Add Caching Layer Modernization	10
VI.	Best Technology Recommendations – With Pros & Cons	12
	Option 1: Stay on Flask but Modularize	12
	Option 2: Move from Flask to FastAPI Modernization	13
	Option 3: Move Backend to Elasticsearch for Read Model Modernization	14
	Option 4: Consider Production-grade NoSQL Modernization	15

TABLE OF CONTENTS

ABOUT THIS DOCUMENTATION

I. About this Documentation

DOCUMENTATION OVERVIEW

This documentation is the technical review of PWT Project shared through git path <https://github.com/Kavinfranco-xome/TestRepo>. This is to evaluate the PWT project for its current system behaviors and non-functional evaluations in terms of Technical debts, Bottlenecks, Pain-points traced in the review along with suggestions for its betterment.

SCOPE OF THIS DOCUMENT

This document covers assessment of the PWT project for its current system behaviors and non-functional evaluations to identify Pain-points traced in the review along with suggestions and proposed approaches given in cases with PROs & CONs.

In assessment of the PWT project below are the aspects considered or analyzed:

- Existing solution design approach
- Existing system speedy code review to assert certain technical aspects
- Current system tools & technologies used
- Different non-functional requirements seen

No architecture diagrams or flows are pictured or solution re-structure provided. Since this is a speedy review may require further reviews or may re-evaluation of this document on aspects like code-review, coding-standards, development approaches, deployment aspects, etc separately.

II. Document Structure

1. OVERVIEW OF DOCUMENT STRUCTURE

This document defined in sections to explain the trace of knowledge gathered during the analysis in terms of non-functional aspects into structural manner.

2. STRUCTURE OF DOCUMENT (SECTIONS)

This is the kind of write-up typically used for architecture review documents, technical proposals, or modernization strategies.

- Current System Overview
- Current System Pain-points Analysis
- Technical / Architectural Solutions for each Pain-points
- Recommended Tools / Frameworks (with Pros & Cons)
- Modernization Possibilities
- A final "Proposed Target Architecture"

CURRENT SYSTEM: OVERVIEW

III. Current System: Overview

The current PWT Web application developed with the following technology stack and repo maintained in GitHub along with the technical characteristics:

1. FRONTEND:

- Built using **AngularJS (1.x)** – legacy framework
- Started using **Gulp task runner** (gulp serve)
- Primarily consumes API data and displays read-heavy views

2. BACKEND:

- Developed on **Python Flask**
- Orchestrates business flows and integrates with multiple data sources
- Exposes REST APIs for the AngularJS Frontend to consume

3. DATABASES:

- **SQL Server**
 - Used for master data, transactional data, and ACID-compliant structures
- **MongoDB**
 - Used for inconsistent provider data
 - Suited for schema-less ingestion

4. TECHNICAL CHARACTERISTICS (NF):

- System is **read-heavy** and **write-light**
- The 3rd-party providers may share data in varying formats - causes schema inconsistency challenges
- Approx. 80% of operations are **database reads**
- Multiple provider integrations require flexibility in data storage
- Scaling requirements growing, especially for read throughput
- Two DB systems used, but not fully optimized for each system's strengths
- Credential based authentication system

CURRENT SYSTEM: PAIN-POINTS ANALYSIS

IV. Current System: Pain-points Analysis

Below is the quick list of Technical Debts, Bottlenecks, & Pain Points found/ traced in my research with my suggested architectural fixes followed thereby.

A. TECHNICAL DEBTS: LEGACY ANGULARJS UI TECHNOLOGY

- Difficult to maintain outdated tools
 - No / lack of community support or availability of new features
 - Hard to scale or extend
 - Performance degrading for large datasets
 - Raise of Security vulnerability risks due to aging framework
 - Learning and coder exposure with AngularJS becoming hard
-

B. TECHNICAL DEBTS: FLASK AS MONOLITHIC API LAYER

- All business logic is packed into a single Flask app
 - Complex provider-specific logic mixed with core logic (no separation of features/ components)
 - Hard to scale horizontally without breaking state
 - No clear boundary between read models and write models
 - Read-heavy API puts load on the core DB (traffic & load)
-

C. PAIN POINT: SQL SERVER MAY STRUGGLES WITH SCALING READ LOADS

- SQL Server scales **vertically**, but not horizontally
 - Workload reads (policy-data, claim-data, premium) cause peak load
 - Even read replicas require complex setup
 - Relational schemas struggle with dynamic provider data
-

CURRENT SYSTEM: PAIN-POINTS ANALYSIS

D. BOTTLENECK: MONGODB NO CLEAR DATA SEGREGATION

- MongoDB stores unstructured provider feeds
 - But backend still queries SQL Server for many combined read views
 - No clear “read model” or caching strategy
 - Leads to high latency, duplicated code, and inconsistent performance
 - MongoDB may not be indexed properly
-

E. PAIN POINT: LACK OF CACHING & NO READ / WRITE SEPARATION

- UI repeatedly hits APIs without any request optimization
 - APIs repeatedly hit SQL Server (it cost performance penalty), unnecessary DB load for master / adequate data (Caching may help, but further review require)
 - May expensive aggregations done on each request
 - Read-heavy system not optimized for serving read / write separation like CQRS
 - Hard to scale API, DB, UI independently
 - Current architecture is not future-proof (outdated for even maintenance, clean-code, etc)
 - Can use message queue or asynchronous pipeline for provider data (Further research require)
-

TECHNICAL / ARCHITECTURAL SOLUTIONS FOR EACH PAIN-POINTS

V. Technical / Architectural Solutions for each Pain-points

Below are the suggested Technical / Architectural resolutions for the above list of Technical Debts, Bottlenecks, & Pain Points traced during my research.

RESOLUTION A — REPLACE ANGULARJS WITH SUITABLE UI FRAMEWORK

SUGGESTED UI FRAMEWORKS

FRAMEWORK	PROS	CONS	REMARK
React.js	Huge ecosystem, flexible, very popular (can use TypeScript)	More decisions - more boilerplate, UI templates	
Angular 17+	Strong enterprise support, structured, TypeScript based	Heavier learning curve, code maintainability	
Vue 3	Simple, elegant, easy to learn, reactive design	Smaller enterprise ecosystem	
ASP.Net Core Blazor	Integrative with Backend, More control, More UI controls & templates	Code maintainability, learning curve, same developer have to complete the full stack development	

RECOMMENDATION

React or Angular 17 for long-term enterprise-grade UI, otherwise ASP.Net Core Blazor for native feel & advantages.

ADVANTAGES / GAINS

- Faster rendering (virtual DOM)
- Better state management
- TypeScript support
- Modern tooling (Vite, Webpack)
- Long-term maintainability
- In case of ASP.Net Core Blazor - Tight relationship between UI-Backend
- Testability and code maintainability

TECHNICAL / ARCHITECTURAL SOLUTIONS FOR EACH PAIN-POINTS

RESOLUTION B — BREAK FLASK MONOLITH TO MICROSERVICES OR MODULAR MONOLITH

RECOMMENDATION

Use a **modular monolith** or **microservices** approach based on scale.

SUGGESTED SEGMENTATION

1. Provider Integration Service
2. Aggregation Service
3. Core Business API (transactional) and Query service APIs
4. Read Model Service (MongoDB/Redis optimized for UI reads)

TOOLS

1. FastAPI (much faster, async)
2. Flask Blueprints (modular monolith)
3. Message queues (RabbitMQ / Kafka) based on demand

ADVANTAGES / GAINS

- Better scaling, clear responsibility boundaries
 - Long-term maintainability
-

TECHNICAL / ARCHITECTURAL SOLUTIONS FOR EACH PAIN-POINTS

RESOLUTION C — READ / WRITE MODERNIZATION

RECOMMENDATION

Change the backend architecture for reads and writes separately using CQRS Pattern with Read Models.

SUGGESTED APPROACH

1. Write & ACID: SQL Server
2. Re-write the backend architecture using CQRS Pattern
3. Read model: MongoDB or Elastic Search (Offload Reads From SQL Server)
4. High-speed caching: Redis (suggested)
5. Read Model Service (MongoDB/Redis optimized for UI reads)

ADVANTAGES / GAINS

- Due to faster Reads, SQL load drops dramatically
 - Dynamic provider structures stored naturally in MongoDB
 - Aggregations precomputed for the UI
-

TECHNICAL / ARCHITECTURAL SOLUTIONS FOR EACH PAIN-POINTS

RESOLUTION D — USE MONGODB PROPERLY FOR VARIABLE DATA

RECOMMENDATION

Use mongo dB properly for variable data, follow DB Optimization technics and apply proper indexing & rethink schema guidelines.

SUGGESTED DB OPTIMIZATION & SCHEMA GUIDELINES

1. Use schema validation for partial enforcement
2. Use bucketed collections for large volumes
3. Use Atlas Search for faster text queries
4. Store each provider feed in provider-specific collections

ADVANTAGES / GAINS

- Faster queries
 - Less code complexity in Flask
 - MongoDB becomes primary source for read-heavy flows
 - Long-term maintainability
-

TECHNICAL / ARCHITECTURAL SOLUTIONS FOR EACH PAIN-POINTS

RESOLUTION E — ADD CACHING LAYER MODERNIZATION

RECOMMENDATION

Change the backend architecture for Caching frequently asked / master data using Redis Catch.

SUGGESTED APPROACH

Use Catch system for frequently asked data locally in the server, bellow are some best areas:

1. Session caching
2. Precomputed query caching
3. API response caching
4. Rate limiting
5. Hot Data Layer

TOOLS

1. Redis (much faster, catching)
2. Catch configurations

ADVANTAGES / GAINS

- Reads from DB drop drastically
 - UI becomes significantly faster
 - Cheaper scaling vs. database scaling
-

TECHNICAL / ARCHITECTURAL SOLUTIONS FOR EACH PAIN-POINTS

SUGGESTIONS —

RECOMMENDATION

For Provider data -> use MQ/ Kafka -> for pre Process MongoDB/SQL -> Read models updated

TOOLS (ANY)

1. RabbitMQ
2. Kafka

ADVANTAGES / GAINS

- Modern approach for the Decoupled ingestion
 - Modern Event-driven processing
 - No slow external API calls blocking the App
-

BEST TECHNOLOGY RECOMMENDATIONS – WITH PROS & CONS

VI. Best Technology Recommendations – With Pros & Cons

Based on different analysis and review below are some of the Technology Recommendations to consider, along with their Pros & Cons also listed for your review.

OPTION 1: STAY ON FLASK BUT MODULARIZE

TECHNOLOGY RECOMMENDATIONS

Based on current perspective, not to go for major change then stay on Flask backend, but update with latest recommendations and modularizations. Follow code quality and coding reviews.

PROS:

- Simple and familiar
- Reuse existing code
- Easy to deploy
- Low rewrite effort
- Tested and confirmed code

CONS:

- Limited async support
 - Scalability limited compared to FastAPI
-

BEST TECHNOLOGY RECOMMENDATIONS – WITH PROS & CONS

OPTION 2: MOVE FROM FLASK TO FASTAPI MODERNIZATION

TECHNOLOGY RECOMMENDATIONS

Based on current perspective, have bandwidth for change then go out from Flask backend, to FastAPI a modernization approach.

PROS:

- Async-first (better for I/O heavy reads)
- Modern Python framework
- Auto documentation via OpenAPI
- Better performance
- Better scalability
- Easy to deploy

CONS:

- Migration effort (however, analyze the migration complexity of no. components & pages)
 - Requires async programming practices
-

BEST TECHNOLOGY RECOMMENDATIONS – WITH PROS & CONS

OPTION 3: MOVE BACKEND TO ELASTICSEARCH FOR READ MODEL MODERNIZATION

TECHNOLOGY RECOMMENDATIONS

Based on current perspective, have bandwidth for change then go out from Flask backend to Elastic-search for read model modernization approach.

PROS:

- Highly optimized search queries
- Best for logs, analytics, dashboards, provider feed searches
- Better performance

CONS:

- Migration effort (however, analyze the migration complexity of no. components & pages)
 - Not ACID
 - Additional operational overhead
-

BEST TECHNOLOGY RECOMMENDATIONS – WITH PROS & CONS

OPTION 4: CONSIDER PRODUCTION-GRADE NOSQL MODERNIZATION

TECHNOLOGY RECOMMENDATIONS

Based on current perspective, have bandwidth for change then go for NoSQL which are familiar now-a-days a modernized approach.

Comparatives study – generic thinking not to stick with MongoDB alone

DB	BEST FOR	PROS	CONS
MongoDB	Document-based feeds	Easy scaling, JSON-native	Needs careful indexing
Cassandra	Heavy write throughput	Extremely scalable	Not consistent enough for banking
Couchbase	KV + Document	Great read speeds	Less community adoption

PROS:

- Optimized queries
- Scalable

CONS:

- Migration effort (for other DB)
 - Additional operational overhead
-